

Exploring Performance–Efficiency Trade-offs in Semantic Segmentation on the Cityscapes Dataset

Yusuf Berkan Gokce
Eindhoven University of Technology
Artificial Intelligence and Engineering Systems
Eindhoven, Netherlands
y.b.gokce@student.tue.nl

Abstract—Semantic segmentation requires assigning a class label to each pixel in an image and is widely used for detailed scene understanding. While modern deep learning models achieve strong segmentation performance, they often involve high computational cost, limiting their efficiency. In this work, we investigate the trade-off between segmentation accuracy and computational efficiency on the Cityscapes dataset. A U-Net model is used as a baseline, while DeepLabV3 with ResNet50 and ResNet101 backbones are evaluated for peak performance, and ENet is considered as a lightweight model for efficiency. The results show that DeepLabV3 with ResNet101 achieves the highest segmentation accuracy, whereas ENet significantly reduces computational cost and model size while achieving the highest efficiency score. However, this efficiency comes with reduced segmentation detail. These findings highlight the inherent trade-off between accuracy and efficiency and demonstrate the importance of selecting models based on specific performance and resource requirements.

Index Terms—semantic segmentation, DeepLabV3, ENet, U-Net, model efficiency, performance trade-off, Cityscapes dataset, neural network architectures

I. INTRODUCTION

Semantic segmentation is a core task in computer vision where each pixel in an image is assigned a class label. Unlike image classification, which gives one label per image, segmentation provides detailed information about the scene. This is especially important in applications such as autonomous driving, where it is necessary to distinguish between roads, pedestrians, vehicles, and other objects.

Although deep learning has improved segmentation performance significantly, the task is still challenging. Datasets like Cityscapes contain complex urban scenes with occlusions, varying object sizes, and class imbalance, making accurate prediction difficult. Earlier work such as Fully Convolutional Networks (FCNs) enabled end-to-end pixel-wise prediction [1]. Later, models such as DeepLab improved performance by capturing more contextual information using techniques like atrous convolution [2]. More recent approaches further refine spatial details and achieve strong performance on challenging benchmarks.

However, these improvements usually come with increased computational cost. High-performing models such as DeepLab-based architectures rely on deep backbones and complex operations, making them computationally expensive and slower at inference time [2]. In contrast, lightweight

architectures such as ENet are designed for efficiency and real-time performance, but typically achieve lower accuracy [3].

This creates a fundamental trade-off between segmentation accuracy and computational efficiency. While larger models aim to maximize performance, smaller models prioritize speed and efficiency, which can be more suitable for real-world deployment.

In this work, we analyze this trade-off on the Cityscapes dataset by evaluating a set of commonly used segmentation architectures with different design priorities, including both high-capacity models and lightweight real-time networks. Specifically, we investigate the following research question:

What is the trade-off between segmentation accuracy and computational efficiency across different semantic segmentation architectures on the Cityscapes dataset?

To answer this question, we compare models using both performance and efficiency metrics, including Dice score, Intersection-over-Union (IoU), computational cost (GFLOPs), and inference speed (FPS). Through this analysis, we aim to provide insights into how different model choices affect the balance between accuracy and efficiency.

II. METHODOLOGY

A. Dataset

All models are trained and evaluated on the Cityscapes dataset, which contains high-resolution urban street images with pixel-level annotations across 19 semantic classes such as roads, buildings, pedestrians, and vehicles [6]. The dataset presents realistic challenges including scale variation, occlusions, and class imbalance.

In this project, the dataset was already split into training and validation sets. The test set was not directly accessible during development and evaluation; instead, it was hosted on an external submission server. Final model performance on the test set was therefore obtained only through submission, ensuring a fair and unbiased evaluation.

B. Baseline Model

1) *U-Net*: For both benchmarks, a U-Net architecture is used as the baseline. As shown in Figure 1, U-Net follows

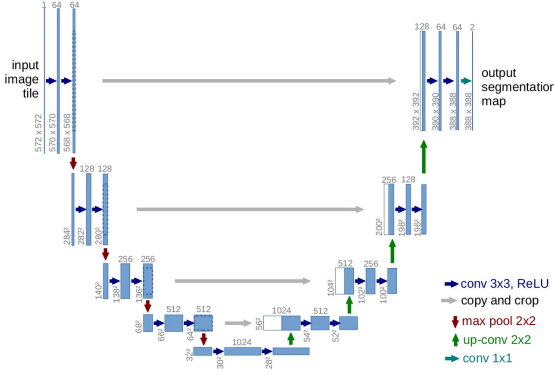


Fig. 1. U-Net architecture consisting of encoder-decoder structure with skip connections [4].

a symmetric encoder-decoder structure with skip connections between corresponding levels [4].

The encoder progressively reduces spatial resolution through convolution and max-pooling while increasing feature channels, allowing the model to learn hierarchical feature representations. The decoder then restores spatial resolution through upsampling and combines encoder features via skip connections. These skip connections are essential, as they preserve fine-grained spatial details that are typically lost during downsampling [4].

U-Net is chosen as a baseline in this study due to its simplicity, relatively low implementation complexity, and strong performance for segmentation tasks. It provides a reliable reference point for both accuracy and efficiency, making it suitable for evaluating improvements introduced by more advanced architectures.

C. Peak Performance Models

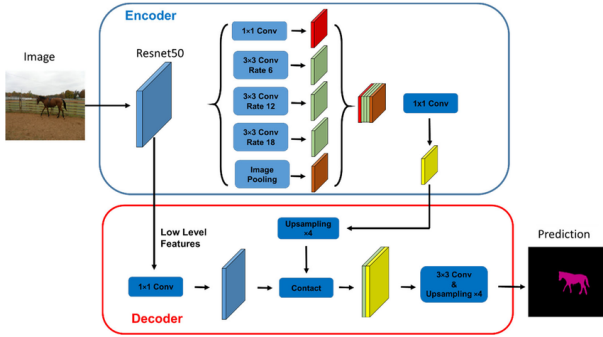


Fig. 2. The network structure combining Deeplabv3 and ResNet50 [5].

1) *DeepLabV3 with ResNet50*: To improve segmentation accuracy beyond the U-Net baseline, DeepLabV3 is used as a more advanced architecture. As shown in Figure 2, DeepLabV3 introduces several architectural improvements that enhance performance, particularly in complex scenes.

The model uses a ResNet50 backbone as a feature extractor [5]. This backbone is pretrained on ImageNet and then fine-tuned on the Cityscapes dataset, allowing the model to benefit

from strong initial feature representations while adapting to the target domain.

A key component of DeepLabV3 is the Atrous Spatial Pyramid Pooling (ASPP) module. ASPP applies multiple parallel dilated convolutions with different dilation rates, enabling the model to capture contextual information at multiple scales [2]. This is particularly important in semantic segmentation, where objects can appear at varying sizes.

Compared to U-Net, DeepLabV3 improves performance by:

- capturing larger receptive fields without reducing resolution
- incorporating multi-scale context through ASPP
- leveraging deep pretrained backbones for feature extraction

These properties make DeepLabV3 with ResNet50 a strong candidate for achieving higher segmentation accuracy.

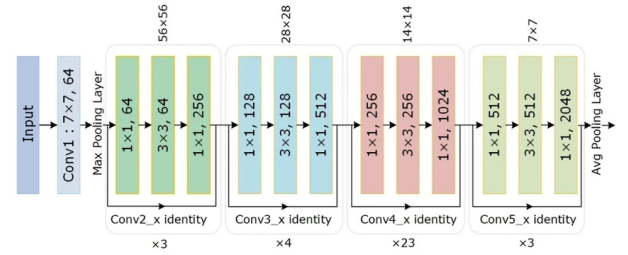


Fig. 3. ResNet101 architecture with residual bottleneck blocks [7].

2) *DeepLabV3 with ResNet101*: To further improve performance, the ResNet50 backbone is replaced with a deeper ResNet101 network [5]. The structure of ResNet101 is shown in Figure 3.

ResNet101 contains significantly more residual bottleneck blocks compared to ResNet50, allowing it to learn more complex and abstract feature representations. Like the ResNet50 version, pretrained weights are used and the model is fine-tuned on Cityscapes.

The increased depth provides:

- a larger effective receptive field
- improved representation of fine details
- better performance on challenging classes and dense scenes

ResNet101 backbone, when combined with the DeepLabV3 framework, becomes a strong candidate for achieving higher segmentation accuracy in the peak-performance benchmark.

D. Efficiency-Oriented Models

1) *ENet*: To optimize for efficiency, ENet is selected as a lightweight segmentation model designed for real-time applications [3]. Its architecture is illustrated in Figure 4.

Unlike traditional segmentation models, ENet is specifically designed to reduce computational cost while maintaining reasonable performance.

The initial block, shown in Figure 4a, processes the input through two parallel paths: a convolution with stride 2 and a max-pooling operation. The outputs are then concatenated.

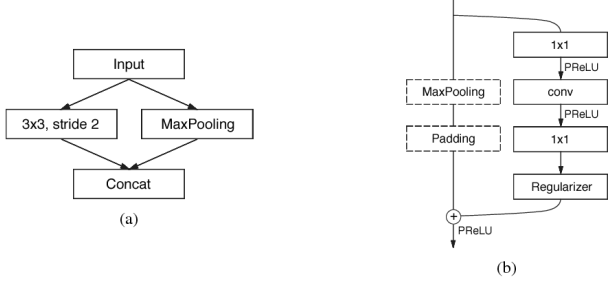


Fig. 4. ENet architecture showing initial block (a) and bottleneck module (b) [3].

This early reduction in spatial resolution lowers the computational cost of subsequent layers while preserving essential information.

The main building component of ENet is the bottleneck module, shown in Figure 4b. It follows a residual structure and consists of three stages:

- a 1×1 projection layer to reduce the number of channels
- a main convolution layer (regular, dilated, or asymmetric)
- a 1×1 expansion layer to restore the dimensions

ENet achieves efficiency through several key design choices:

- early downsampling to reduce feature map size
- use of 1×1 convolutions to limit computation
- asymmetric convolutions (e.g., 5×1 and 1×5) instead of larger kernels
- dilated convolutions to increase the receptive field without additional cost

Another important design choice is the use of a large encoder and a relatively small decoder [3]. As a result, most computation is concentrated in feature extraction, while the upsampling stage remains lightweight.

Overall, these design choices make ENet significantly more efficient than U-Net in terms of model size, computational cost (GFLOPs), and memory usage, while still achieving reasonable segmentation performance. This makes it a strong candidate for the efficiency benchmark.

E. Evaluation Metrics

For the peak performance benchmark, the main evaluation metric is the Mean Dice Score:

$$Dice = \frac{2TP}{2TP + FP + FN} \quad (1)$$

Intersection-over-Union (IoU) is also used:

$$IoU = \frac{TP}{TP + FP + FN} \quad (2)$$

For the efficiency benchmark, additional metrics are considered:

- GFLOPs
- Model size
- Inference speed (FPS)

An efficiency metric is defined as:

$$Efficiency = \frac{Dice}{TFLOPs} \quad (3)$$

III. RESULTS

This section presents the quantitative and qualitative results of the evaluated models on the Cityscapes test set. All results were obtained through submission to the evaluation server.

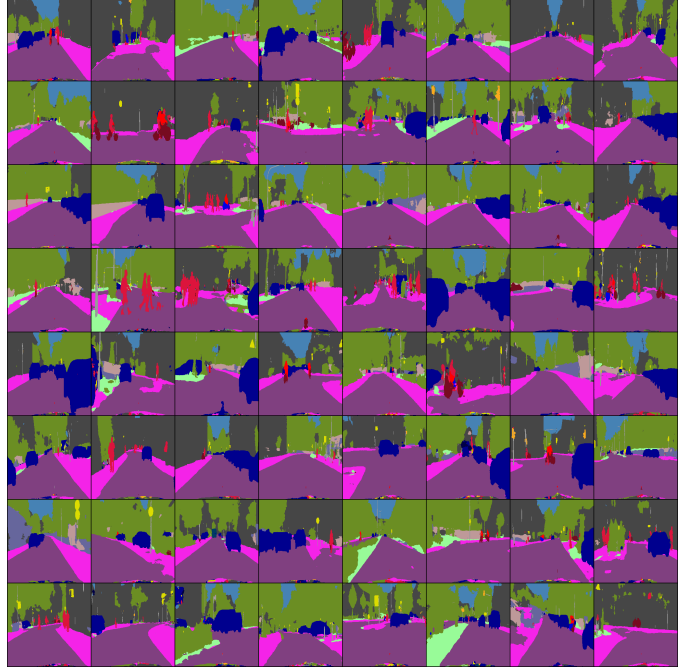


Fig. 5. Segmentation outputs of the U-Net baseline model.

Figure 5 shows segmentation outputs of the U-Net baseline model. The model captures the overall structure of the scene but shows reduced precision in object boundaries and smaller classes.

Table I summarizes the quantitative results for the peak-performance benchmark. Both DeepLabV3 models outperform the U-Net baseline in terms of Mean Dice and Mean IoU, with the ResNet101 variant achieving the highest scores.

Table II presents the efficiency benchmark results. ENet significantly reduces computational cost and model size while achieving the highest efficiency score, whereas the U-Net baseline requires substantially higher computation for comparable accuracy.

Figures 6 and 7 show qualitative segmentation outputs of the peak-performance and efficiency-oriented models. The DeepLabV3 with ResNet101 model produces more refined and detailed segmentations, particularly around object boundaries, while ENet preserves the overall structure of the scene with reduced detail.

IV. DISCUSSION

This study examined the trade-off between segmentation accuracy and computational efficiency across different architectures on the Cityscapes dataset. The results show a clear

TABLE I
PEAK PERFORMANCE COMPARISON ON THE CITYSCAPES TEST SET

Model	Mean Dice	Mean IoU	Flat	Cons.	Obj.	Nat.	Sky	Human	Veh.
DeepLabV3 (ResNet101)	0.5600	0.4569	0.6749	0.8092	0.3265	0.3653	0.0666	0.3546	0.3474
DeepLabV3 (ResNet50)	0.5506	0.4458	0.6626	0.7853	0.3000	0.3474	0.0812	0.3468	0.3826
U-Net (Baseline)	0.4755	0.3882	0.6027	0.7907	0.1883	0.1533	0.0870	0.1766	0.2776

TABLE II
EFFICIENCY COMPARISON ON THE CITYSCAPES TEST SET

Model	Dice/TFLOPs	Size (MB)	GFLOPs	FPS	Mean Dice	Mean IoU
ENet	15.4767	1.6032	29.0487	46.9100	0.4496	0.3686
U-Net (Baseline)	0.1855	65.9340	2563.5586	8.8600	0.4755	0.3882

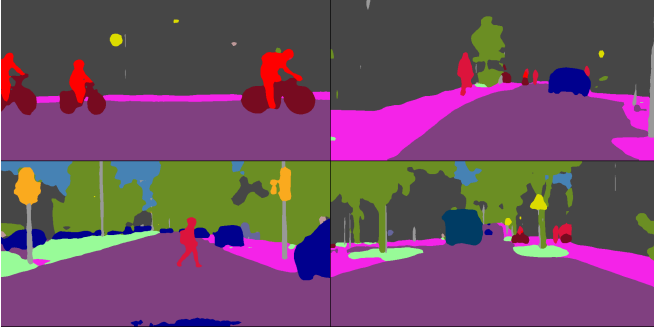


Fig. 6. Segmentation outputs of DeepLabV3 with ResNet101.

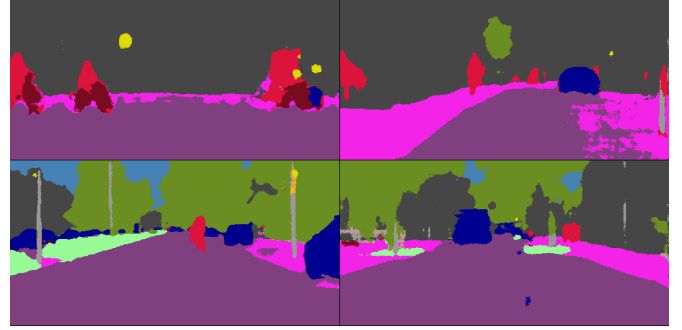


Fig. 7. Segmentation outputs of ENet.

separation between models optimized for accuracy and those optimized for efficiency.

DeepLabV3 with ResNet101 achieves the highest performance, improving significantly over the U-Net baseline in both Mean Dice and Mean IoU. This improvement is mainly due to its deeper backbone and the use of the ASPP module, which allows better multi-scale feature extraction. The qualitative results also show more precise object boundaries and better handling of smaller objects. However, this performance comes with increased computational cost, making such models less suitable for real-time or resource-limited applications.

On the other hand, ENet demonstrates strong efficiency by drastically reducing model size and GFLOPs. It achieves a higher efficiency score while maintaining reasonable segmentation performance. This is enabled by design choices such as early downsampling and lightweight bottleneck modules. However, ENet produces less detailed segmentations, especially around object boundaries, indicating a loss in fine-grained accuracy.

These results confirm that segmentation accuracy and computational efficiency are competing objectives. High-capacity models provide better accuracy but require significantly more computation, while lightweight models are more efficient but sacrifice detail.

This work has some limitations. Only a limited set of architectures was evaluated, and more recent models may offer better trade-offs. In addition, experiments were conducted on a single dataset, which may limit generalization.

Future work could explore hybrid or transformer-based

models, as well as model compression techniques such as pruning and quantization to improve efficiency without large performance loss.

Overall, the results highlight the importance of selecting models based on application requirements, balancing accuracy and efficiency depending on the deployment scenario.

V. CODE AVAILABILITY

The implementation and experimental code used in this work are publicly available at: <https://github.com/BerkanGokce/NNCV.git>

REFERENCES

- [1] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *CVPR*, 2015.
- [2] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Re-thinking Atrous Convolution for Semantic Image Segmentation," arXiv:1706.05587, 2017.
- [3] A. Paszke et al., "ENet: A Deep Neural Network Architecture for Real-Time Semantic Segmentation," arXiv:1606.02147, 2016.
- [4] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.
- [5] L. Kou, M. Sysyn, S. Fischer, J. Liu, and O. Nabochenko, "Optical Rail Surface Crack Detection Method Based on Semantic Segmentation Replacement for Magnetic Particle Inspection," *Sensors*, vol. 22, no. 21, p. 8214, 2022.
- [6] M. Cordts et al., "The Cityscapes Dataset for Semantic Urban Scene Understanding," in *CVPR*, 2016.
- [7] S. Albahli and T. Nazir, "AI-CenterNet CXR: An artificial intelligence (AI) enabled system for localization and classification of chest X-ray disease," *Frontiers in Medicine*, vol. 9, 2022, doi: 10.3389/fmed.2022.955765.